

Proyecto 1 — Informe Final

Clasificación de Contaminaciones en una Línea de Producción Simulada

Walther Alejandro Barrantes Yan

C31037

1. Introducción

El objetivo de este proyecto fue construir, desde cero, un sistema capaz de detectar granos de arroz en imágenes tomadas sobre una superficie blanca. La idea viene del contexto de inspección visual en líneas de producción: en la industria es común que existan partículas o contaminantes que no deberían estar ahí, y detectarlos manualmente es tedioso, lento y propenso a errores. Automatizar ese proceso con aprendizaje automático es una solución natural.

Para este proyecto se simuló una línea de producción muy sencilla: una hoja blanca haciendo de superficie y granos de arroz como contaminantes. Las imágenes donde aparece arroz son ejemplos positivos; las que muestran otros objetos (como clips) o la superficie vacía son ejemplos negativos. Con ese conjunto de datos se entrenaron y compararon cuatro algoritmos de clasificación clásica: Árbol de Decisión, Naive Bayes, KNN y SVM.

1.1. ¿Por qué clasificación binaria?

El problema se reduce a una pregunta simple: ¿hay arroz en esta imagen o no? Eso es exactamente lo que resuelve un clasificador binario, que asigna a cada imagen una de dos etiquetas: positivo (etiqueta 1, con arroz) o negativo (etiqueta 0, sin arroz). Este enfoque es directo y encaja perfectamente con los algoritmos clásicos de aprendizaje automático supervisado.

1.2. Algoritmos evaluados

Se probaron cuatro algoritmos con enfoques muy distintos entre sí, lo que permite entender qué tipo de método funciona mejor para este problema en particular.

Tabla 1: Algoritmos de clasificación evaluados.

Algoritmo	¿Cómo funciona?
Árbol de Decisión	Hace preguntas del tipo “¿el píxel X es blanco o negro?” y va dividiendo los datos hasta llegar a una clasificación.
Naive Bayes	Usa probabilidades para estimar qué tan probable es que una imagen sea positiva dado el valor de sus píxeles.
KNN	Busca las K imágenes más parecidas en el conjunto de entrenamiento y vota por la clase más común entre ellas.
SVM	Encuentra la frontera matemática que mejor separa las imágenes con arroz de las que no tienen.

2. Recolección de Datos

2.1. Cómo se simuló la línea de producción

Para generar el dataset se usó una hoja blanca como superficie y se fotografiaron dos tipos de escenas: una con granos de arroz dispersos (contaminación positiva) y otra con clips de oficina o sin nada encima (contaminación negativa). Las fotos las tomé con un iPhone en formato HEIC, sin ningún tipo de control especial de iluminación ni soporte para la cámara. Las condiciones variaron entre tomas: distancia, ángulo y luz cambiaron naturalmente, lo que hace el dataset más realista pero también más difícil para los algoritmos.

2.2. Tamaño y composición del dataset

El conjunto de datos quedó con **120 imágenes** en total, perfectamente balanceadas entre clases:

Tabla 2: Composición del dataset.

Clase	Etiqueta	Cantidad
Con arroz (positivo)	1	60
Sin arroz (negativo)	0	60
Total		120

Tener exactamente 50 % por clase es importante porque si el dataset estuviera desbalanceado, un modelo podría obtener buena exactitud simplemente prediciendo siempre la clase mayoritaria, sin haber aprendido nada real. Con el balance perfecto, cualquier modelo que ronde el 50 % de exactitud básicamente está tirando una moneda al aire.

2.3. Distribución y muestra visual

La Figura 1 confirma gráficamente el balance del dataset, y la Figura 2 muestra cómo lucen algunas imágenes después del preprocesamiento.

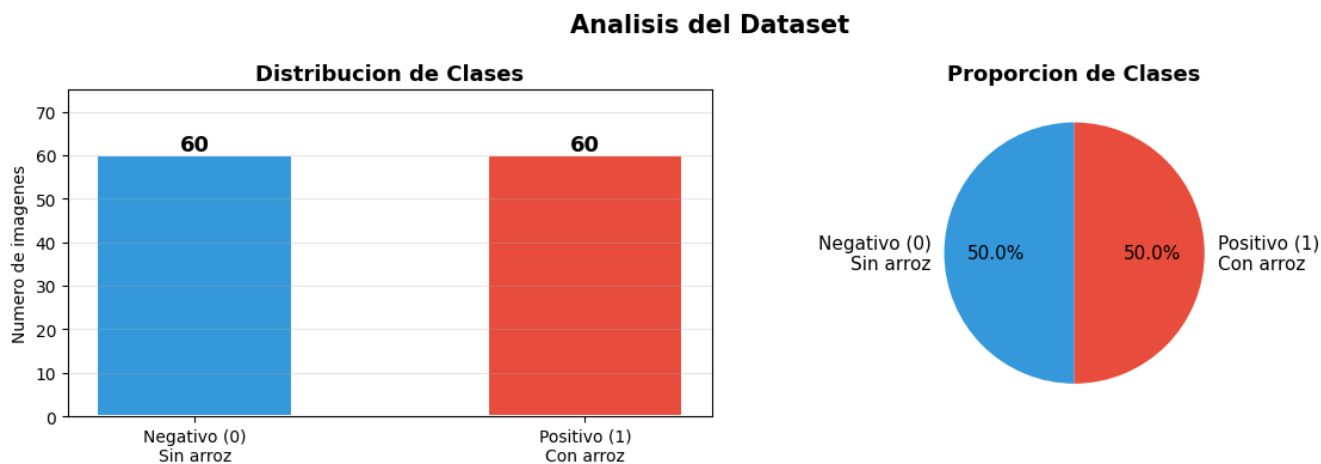


Figura 1: Distribución de clases del dataset. 60 imágenes positivas (con arroz) y 60 negativas, balance perfecto del 50 %.



Figura 2: Muestra representativa del dataset ya preprocesado. Fila superior: imágenes positivas con granos de arroz visibles. Fila inferior: imágenes negativas con clips u otras superficies sin arroz.



Figura 3: Ejemplo positivo: superficie con granos de arroz. Los píxeles oscuros marcan la presencia del objeto.

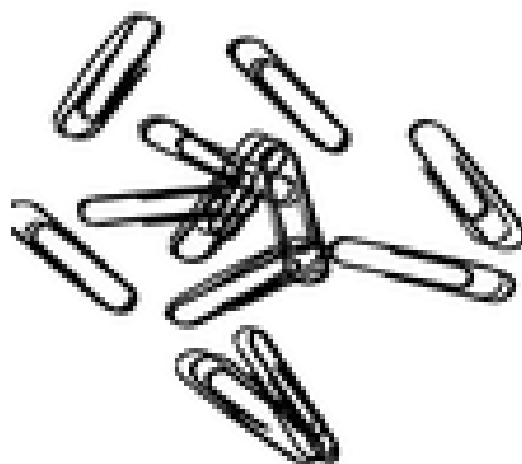


Figura 4: Ejemplo negativo: superficie sin arroz. El fondo predominantemente blanco indica ausencia de contaminación.

3. Preprocesamiento e Ingeniería de Datos

Los algoritmos de clasificación clásica no trabajan con archivos de imagen directamente: necesitan números. El preprocesamiento fue el proceso de convertir cada foto en un vector numérico que los modelos pudieran entender.

3.1. De foto a imagen en blanco y negro

Las fotos originales estaban en formato HEIC y en color. El primer paso fue convertirlas a **escala de grises** y redimensionarlas a **128×128 píxeles**. El color no aporta información relevante para distinguir arroz de clips lo que importa es la forma y presencia de objetos, así que eliminarlo simplifica el problema sin perder nada útil. El tamaño de 128×128 es un balance razonable: suficiente resolución para ver los granos, pero manejable para que los algoritmos clásicos corran en tiempos razonables.

3.2. Binarización

Una imagen en escala de grises tiene píxeles con valores de 0 a 255. Para simplificar aún más la representación, se aplicó un umbral fijo: si el valor del píxel era mayor a 127 se codificó como 1 (fondo blanco), si era 127 o menos se codificó como 0 (presencia de un objeto).

$$p_{\text{binario}} = \begin{cases} 1 & \text{si } p > 127 \\ 0 & \text{si } p \leq 127 \end{cases} \quad (1)$$

Esto reduce el espacio de representación de 256 niveles a solo dos, eliminando variaciones menores de iluminación que no tienen que ver con la presencia o ausencia de arroz.

3.3. Vectorización y construcción del CSV

Una imagen de 128×128 es una matriz. Para que los algoritmos puedan trabajar con ella se “aplana” en un vector de $128 \times 128 = 16,384$ valores. Cada imagen queda representada como una fila en el CSV final, con 16,384 columnas de píxeles más una columna de etiqueta (1 o 0).

```
1 from PIL import Image
2 import numpy as np
3
4 img = Image.open("foto.heic").convert("L")           # escala de grises
5 img = img.resize((128, 128), Image.LANCZOS)          # redimensionar
6
7 array = np.array(img)
8 binario = (array > 127).astype(int)                  # binarizar
9
10 vector = binario.flatten()    # 16384 valores, listo para el CSV
```

Listing 1: Fragmento del preprocesamiento en Python.

El etiquetado fue automático según el nombre del archivo: los archivos con prefijo 1.N.png se marcaron como negativos (etiqueta 0) y los de prefijo 2.N.png como positivos (etiqueta 1). Esto evitó errores de anotación manual.

4. Entrenamiento de Modelos

4.1. Configuración general

El dataset se dividió en 80 % entrenamiento (96 imágenes) y 20 % prueba (24 imágenes), de forma estratificada para mantener el balance de clases en ambas particiones. Para elegir los mejores hiperparámetros de cada modelo se usó **GridSearchCV con 5 pliegues**: básicamente se prueban todas las combinaciones posibles de parámetros y se elige la que da mejor exactitud promedio en validación cruzada.

Para que los resultados no dependieran de una sola partición aleatoria, la evaluación se repitió **20 veces** con distintas semillas. Así se puede reportar exactitud media y desviación estándar, lo que da una imagen más honesta del rendimiento real de cada modelo.

Tabla 3: Parámetros de entrenamiento utilizados.

Parámetro	Valor
División entrenamiento / prueba	80 % / 20 %
Repeticiones de evaluación	20
Pliegues en GridSearchCV	5
Semilla aleatoria	42

4.2. Árbol de Decisión

El árbol de decisión clasifica imágenes haciendo preguntas sobre el valor de píxeles específicos. Por ejemplo: “¿el píxel 14007 vale más de 0.5?”. Según la respuesta va por una rama u otra hasta asignar una clase. Lo interesante de este modelo es que se puede visualizar y entender qué está haciendo exactamente.

Se exploraron 24 combinaciones de hiperparámetros. La mejor configuración fue `criterion='gini'`, `max_depth=10`, `min_samples_split=5`, con una exactitud en validación cruzada del 67.63 % y 62.5 % en prueba. El árbol encontró patrones reales pero con bastante varianza entre iteraciones, lo que sugiere que 120 imágenes son pocas para estabilizarlo.

4.3. Naive Bayes

Naive Bayes calcula, para cada clase, qué tan probable es observar el vector de píxeles de una imagen dada. Asume que cada píxel es independiente de los demás, lo cual es una simplificación fuerte en imágenes, donde píxeles vecinos están evidentemente correlacionados.

Con solo 4 combinaciones de hiperparámetros exploradas, el mejor fue `var_smoothing=0.001`. Fue el más rápido en entrenarse (0.6 segundos), pero sus resultados no superaron el azar, lo que indica que la suposición de independencia lo limita severamente en este tipo de datos.

4.4. KNN

KNN no aprende un modelo explícito. Al clasificar una imagen nueva, busca las K más parecidas en el conjunto de entrenamiento (por distancia euclidiana entre vectores de píxeles) y vota por la clase más común entre ellas.

El problema en este caso es que cada imagen tiene 16,384 dimensiones. En espacios tan grandes, la distancia euclidiana pierde sentido: todas las imágenes quedan aproximadamente igual de lejos entre sí, así que los “vecinos más cercanos” no son realmente representativos. Este fenómeno se conoce como la maldición de la dimensionalidad. El mejor resultado fue con `n.neighbors=3` y distancia euclidiana, pero igualmente quedó en el nivel del azar.

4.5. SVM

SVM busca el hiperplano que separa con el mayor margen posible las imágenes positivas de las negativas. Con el **kernel RBF** puede capturar fronteras de decisión no lineales, transformando implícitamente los datos a un espacio donde las clases sí son separables.

De las 6 combinaciones exploradas, la mejor fue `C=1`, `kernel='rbf'`, con una exactitud del 96.89 % en validación cruzada y 100 % en el conjunto de prueba. El tiempo de búsqueda fue de 9.5 segundos, el más alto de los cuatro modelos, pero completamente razonable.

La Figura 5 muestra la visualización del árbol de decisión, donde se puede ver cómo el modelo usa píxeles como puntos de corte.

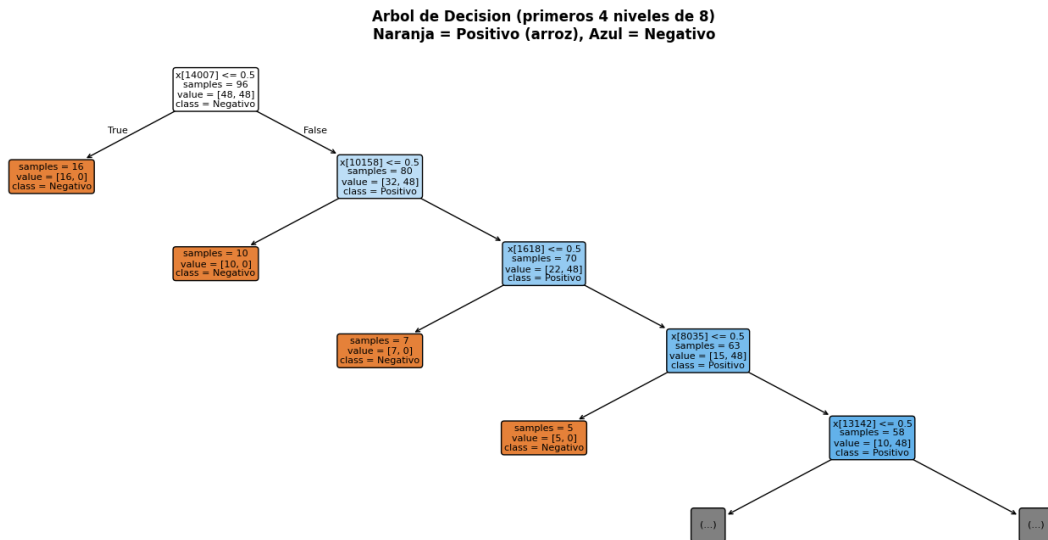


Figura 5: Árbol de decisión entrenado. Cada nodo muestra el píxel evaluado y el índice Gini de impureza. El nodo raíz parte por el píxel 14,007, que resultó ser el más discriminativo entre las dos clases.

5. Resultados

5.1. Métricas utilizadas

Para comparar los modelos se usaron cuatro métricas estándar en clasificación binaria. Vale la pena explicarlas porque cada una mide algo diferente:

Exactitud (Accuracy): el porcentaje de imágenes que el modelo clasificó correctamente, sobre el total. Es la más intuitiva pero puede ser engañosa si las clases estuvieran desbalanceadas.

Precisión: de todas las imágenes que el modelo dijo que tenían arroz, ¿qué porcentaje realmente lo tenía? Un valor bajo indica muchas falsas alarmas.

Recall: de todas las imágenes que sí tenían arroz, ¿qué porcentaje detectó el modelo? Un valor bajo implica contaminaciones que pasaron desapercibidas, lo cual en una línea de producción real sería el error más costoso.

F1: combina precisión y recall en un solo número. Es útil cuando ambas importan y no se quiere sacrificar una por la otra.

$$F1 = \frac{2 \cdot \text{Precisión} \cdot \text{Recall}}{\text{Precisión} + \text{Recall}}$$

5.2. Resultados en el conjunto de prueba

La Tabla 4 muestra el desempeño de cada modelo sobre las 24 imágenes de prueba, con la semilla `RANDOM_STATE=42`.

Tabla 4: Resultados en el conjunto de prueba (24 imágenes: 12 positivas + 12 negativas).

Modelo	Acc	Precisión	Recall	F1	TP	TN	FP	FN
SVM	1.000	1.000	1.000	1.000	12	12	0	0
Árbol de Decisión	0.625	0.588	0.833	0.690	10	5	7	2
KNN	0.500	0.500	1.000	0.667	12	0	12	0
Naive Bayes	0.500	0.000	0.000	0.000	0	12	0	12

El resultado del SVM fue perfecto: clasificó las 24 imágenes correctamente, sin ningún error. KNN predijo positivo para todas las imágenes (por eso tiene recall de 1.0 pero precisión de 0.5 y ningún verdadero negativo), mientras que Naive Bayes hizo exactamente lo contrario: predijo negativo para todo. Ambos básicamente ignoraron los datos. El Árbol de Decisión estuvo en un punto intermedio, con 7 falsas alarmas y 2 contaminaciones no detectadas.

5.3. Evaluación robusta sobre 20 iteraciones

La Tabla 5 muestra los resultados promediados sobre 20 particiones distintas, lo que da más confianza en los números reportados.

Tabla 5: Exactitud y F1 medios sobre 20 iteraciones de evaluación.

Modelo	Acc Media $\pm$ Std	Mín	Máx	F1 Media $\pm$ Std
SVM	0.977 $\pm$ 0.038	0.833	1.000	0.979 $\pm$ 0.034
Árbol de Decisión	0.688 $\pm$ 0.064	0.583	0.792	0.744 $\pm$ 0.051
Naive Bayes	0.519 $\pm$ 0.025	0.500	0.583	0.068 $\pm$ 0.088
KNN	0.504 $\pm$ 0.013	0.500	0.542	0.669 $\pm$ 0.006

El SVM se mantiene por encima del 97 % en promedio, con una desviación estándar pequeña que indica estabilidad. En el peor caso de las 20 iteraciones obtuvo 83.3 %, lo cual sigue siendo bastante aceptable. El Árbol de Decisión ronda el 69 % pero con más varianza, y KNN y Naive Bayes siguen sin superar el nivel del azar.

5.4. Análisis gráfico

Las Figuras 6 y 7 complementan las tablas con una vista visual del rendimiento comparativo.

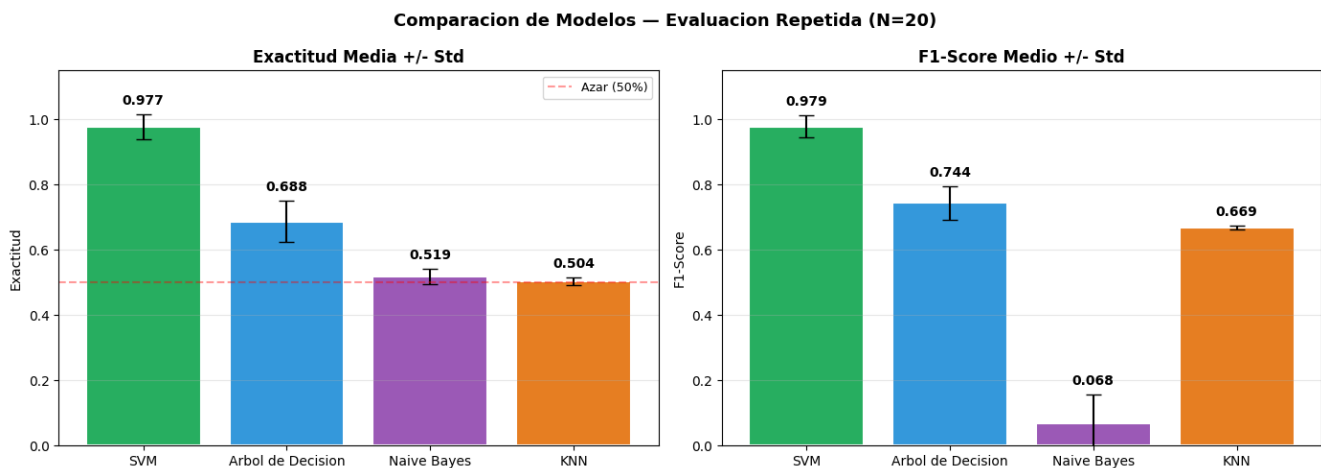


Figura 6: Exactitud y F1 medios de cada modelo con barras de error. La diferencia entre SVM y los demás es muy pronunciada.

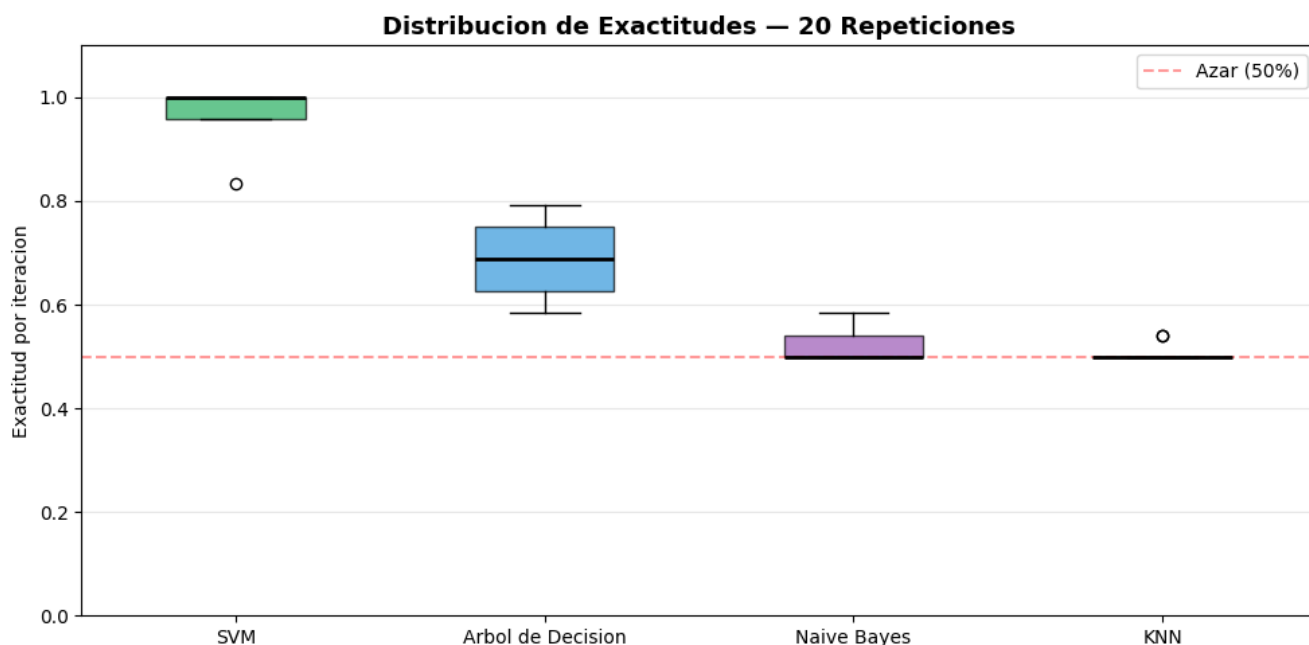


Figura 7: Distribución de exactitudes en las 20 iteraciones. SVM muestra una caja angosta y consistentemente alta. El Árbol de Decisión tiene más dispersión. KNN y Naive Bayes se quedan apilados cerca del 50 %.

Las matrices de confusión de la Figura 8 permiten ver de forma más directa los tipos de errores que comete cada modelo.

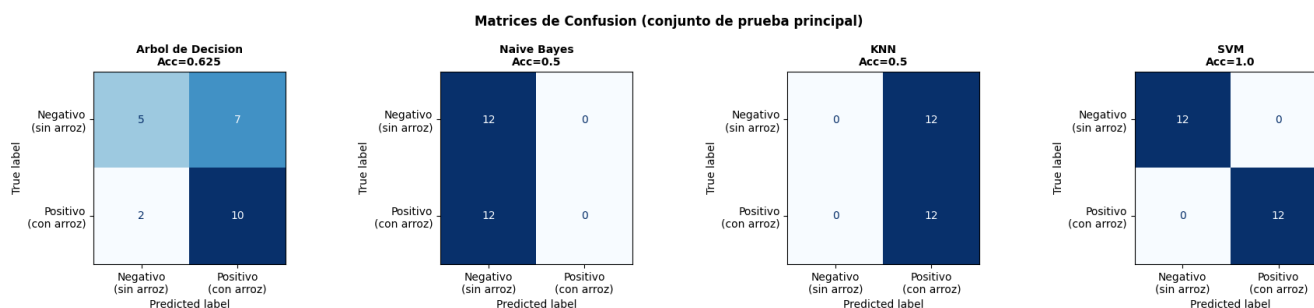


Figura 8: Matrices de confusión para los cuatro modelos. La diagonal principal corresponde a aciertos. SVM tiene solo valores en la diagonal; KNN y Naive Bayes tienen errores sistemáticos en una sola dirección.

5.5. ¿Por qué SVM funciona mejor?

Trabajar con vectores de 16,384 dimensiones es exactamente el tipo de problema donde SVM está en su elemento. El kernel RBF le permite encontrar fronteras de decisión complejas que separan los patrones de píxeles del arroz de los demás, sin sobreajustarse gracias al parámetro de margen. KNN, en cambio, se ve perjudicado porque en tantas dimensiones la distancia euclidiana pierde significado y todos los puntos quedan “igual de lejos”. Naive Bayes trata cada píxel como independiente, lo cual no tiene sentido en una imagen donde los píxeles vecinos tienen valores correlacionados.

5.6. Modelo exportado

El SVM ganador se exportó con joblib bajo el nombre `C31037_Walther_Barrantes.joblib`, reentrenado sobre el 100 % del dataset. Para usarlo en predicción:

```
1 import joblib, numpy as np, cv2
2
3 modelo = joblib.load("models/C31037_Walther_Barrantes.joblib")
4
5 img = cv2.imread("imagen_nueva.png", cv2.IMREAD_GRAYSCALE)
6 img = cv2.resize(img, (128, 128), interpolation=cv2.INTER_NEAREST)
7 vector = (img > 127).astype(int).flatten().reshape(1, -1)
8
9 pred = modelo.predict(vector)
10 print("Con arroz" if pred[0] == 1 else "Sin arroz")
```

Listing 2: Inferencia con el modelo SVM exportado.

6. Conclusiones

Los resultados de este proyecto dejan varias lecciones claras. La más evidente es que no todos los algoritmos clásicos son igualmente útiles para el mismo problema: en este caso la diferencia entre el mejor (SVM con 97.7 % promedio) y los peores (KNN y Naive Bayes rondando el 50 %) es enorme, y tiene sentido matemático explicarlo.

KNN y Naive Bayes fallaron por razones muy concretas. KNN trabaja bien cuando las distancias entre puntos son significativas, pero con 16,384 dimensiones eso deja de ser cierto: todas las imágenes quedan aproximadamente igual de lejos entre sí. Naive Bayes asume que los píxeles son independientes, lo cual es falso en imágenes reales —los píxeles vecinos siempre están correlacionados— y eso lo condena a no aprender la estructura de los datos. Ambos modelos se quedaron atrapados prediciendo siempre la misma clase.

El Árbol de Decisión al menos encontró algo: identificó píxeles discriminativos y construyó reglas que en promedio aciertan el 69 % de las veces. No es suficiente para un sistema de control de calidad real, pero demuestra que hay patrones aprendibles en los datos. Su mayor varianza entre iteraciones sugiere que 120 imágenes son pocas para estabilizarlo.

SVM fue el algoritmo correcto para este problema. Está diseñado para espacios de alta dimensión, y el kernel RBF le permite capturar las diferencias entre las distribuciones de píxeles de las imágenes con y sin arroz. Obtuvo clasificación perfecta en el conjunto de prueba y un promedio de 97.7 % sobre 20 evaluaciones independientes.

Dicho esto, el sistema tiene limitaciones que no se deben ignorar. El dataset es pequeño (120 imágenes) y las condiciones de captura no fueron controladas, por lo que los resultados pueden ser más optimistas de lo que serían en un entorno real. Un conjunto de prueba capturado en condiciones distintas probablemente mostraría un rendimiento menor. Además, la clase negativa está representada solo por clips de oficina, no por las condiciones reales de ausencia de contaminación en una línea de producción. El modelo aprendió a distinguir arroz de clips, que no es exactamente el problema industrial.

Para una siguiente versión del sistema tendría sentido ampliar el dataset, controlar mejor las condiciones de captura, y evaluar el uso de reducción de dimensionalidad como PCA antes de entrenar KNN, que probablemente mejoraría sus resultados considerablemente.

Tabla 6: Resumen de la entrega.

Campo	Valor
Estudiante	Walther Alejandro Barrantes Yan (C31037)
Dataset	120 imágenes $\times$ 16,384 píxeles (128 $\times$ 128)
Balance de clases	60 positivos / 60 negativos
División	80/20 estratificado, evaluado 20 veces
Modelo exportado	SVM con kernel RBF (C=1)
Archivo	<code>C31037_Walther_Barrantes.joblib</code>
Exactitud en prueba	1.000 (100 %)
Exactitud media	0.977 ± 0.038
F1 media	0.979 ± 0.034
Hardware	Google Colab, CPU estándar